

© Brendan Choi 2021

B. Choi, *Introduction to Python Network Automation*

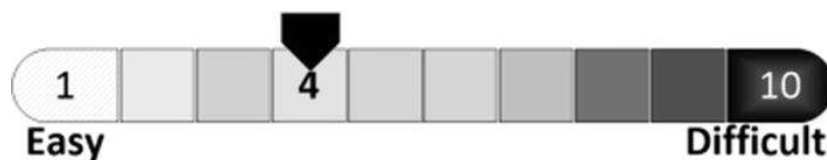
https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-6806-3_3

3. More Python Exercises

Brendan Choi¹

(1) Sydney, NSW, Australia

The Romans borrowed an old Athenian concept and turned it into a proverb: “Repetito master sturdiorum,” which means “Repetition is the mother of all learning.” In the previous chapter, you practiced basic Python concepts, but you will get more practice with more relatable subjects and scenarios in this chapter. Some parts of the exercises will contain content that will be directly relatable to the scripts you will be writing in the second half of this book. When we do each exercise in this chapter, let your fingers do the typing, but let your brain think about how a certain exercise can help you automate parts of your work.



Getting Ready for the Exercises

This book works on the assumption that you are new to network automation using Python. When we look at the day-to-day tasks of a typical IT engineer, systems engineers and network engineers do not have to type as much as DevOps engineers. This is just the nature of the job because DevOps engineers are always on the move, writing code and creating applications as part of various projects. In other words, for most of the non-DevOps IT staff who are reading this book, your time will be spent on installing, configuring, and troubleshooting various issues in your field. It will be challenging to find the time during work hours to practice as you are not paid to learn to code and develop automation applications. Yet, almost all enterprise IT teams are trying to push their IT engineers to start learning a programming language and automating repetitive tasks. Being a good employee and exemplary IT engineer, you have probably been using time outside of work hours to learn Python. Not all of us will understand programmability concepts or learn the Python basics on our first attempt; just be consistent and keep trying until you do. The only way to overcome the challenges is through more exercises. If you have specific Python challenges, please get into the habit of searching for that topic on

Google and reading further about it. Now, let's get some more Python practices using relatable examples.

In Chapter 2, you learned the basics of Python. This chapter will help you practice even more to prepare you for network automation using Python. The more you practice, the better you will get.

Exercise 3-1: Concatenate a List and a Tuple into a Single List

```
>>> fruits = ['apple', 'orange', 'mango']
>>> vegetables = ('broccoli', 'potato', 'spinach')
① >>> favorites = fruits + list(vegetables)
>>> print(favorites)
['apple', 'orange', 'mango', 'broccoli', 'potato',
'spinach']
```

Explanation

Create one variable in the list and create one variable in a tuple

- ① . Convert the tuple to a list using the `list()` method .
Concatenate and merge one list and one tuple into a single list.

Exercise 3-2: Use Python as a Calculator

```
>>> eigrp, ospf, rip = 90, 110, 120
>>> path1, path2, path3 = 3, 6, 9
① >>> admin_distance = (eigrp * path1) + (ospf * path2)
+ (rip * path3)
>>> print(admin_distance)
2010
```

Explanation

- ① Use Python as a calculator and calculate an administrative distance.

Exercise 3-3: Do Some Basic String `format()` Exercises

```
>>> name = 'Hugh'
>>> age = 15
① >>> detail = 'His name is %s and he is %d.'
>>> print(detail %(name, age))
His name is Hugh and he is 15.
```

```

>>> Name, age, height = 'Hugh', 15, 174.5
>>> detail = ('His name is {}, he is {} and {} cm
tall.'.format(name, age, height))
>>> print(detail)
His name is Hugh, he is 15 and 174.5 cm tall.
>>> detail = ('His name is {0}, he is {1} and {2} cm
tall.'.format(name, age, height))
② >>> print(detail)
#1 His name is Hugh, he is 15 and 174.5 cm tall.
#2 >>> detail = ('His name is {name}, he is {age} and
#3 {height} cm tall.'.format(name='Joshua', age=16,
#4 height=178))
>>> print(detail)
His name is Joshua, he is 16 and 178 cm tall.
>>> detail = ('His name is {0}, he is {age} and
{height} cm tall.'.format('Michael', age=12,
height=170))
>>> print(detail)
His name is Michael, he is 12 and 170 cm tall.

>>> person = {'height':174.5, 'name':'Hugh', 'age':15}
>>> print('{name} is {age} years old and {height} cm
tall.'.format(**person))
③ Hugh is 15 years old and 174.5 cm tall.

```

Explanation

- ① This is a basic string (%s) and digit (%d) formatting example .

- ②
- #1. format() example: Default arguments
 - #2. format() example: Positional arguments
 - #3. format() example: Keyword arguments
 - #4. format() example: Mixed arguments

- ③ This is an `str.format(**mapping)` example. This is an example of the argument parsing used in Python, so make a special note of the two asterisks used in the example.

Exercise 3-4: Ask for a Username

Write the code in Notepad++ as shown in Figure [3-1](#).

- ①
- ```

ex3_4.py

print('Please enter your name: ')

name = input()

```

```
print('Thank you, ', name)
```

The screenshot shows a Notepad++ window titled 'C:\Python39\ex3\_4.py - Notepad++'. The code in the editor is:

```
1 # ex3_4.py
2 print('Please enter your name: ')
3 name = input()
4 print('Thank you, ', name)
```

The console window below shows the execution output:

```
C:\Python39\python.exe "C:\Python39\ex3_4.py"
Process started (PID=4364) >>>
Please enter your name:
Michelle
Thank you, Michelle
<<< Process finished (PID=4364). (Exit code 0)
===== READY =====
```

The status bar at the bottom indicates: Python file, length: 90, lines: 4, Ln: 1, Col: 11, Sel: 0 | 0, Windows (CR LF), UTF-8, INS.

**Figure 3-1.** User input, thanking the user

### Explanation

- ① Create some Python code that will ask for the user's name. You are going to use this as the basis of username and password tool development. You will have some fun while asking for someone's name.

### Exercise 3-5: Get a Username: Version 1

Write the code and save it as `ex3_5.py` and run the code in Notepad++. (see Figure 3-2)

- ```
#ex3_5.py
```
- ①
- ```
name = input('Please enter your name: ')
print(f'Hi, {name}.')
```

The screenshot shows a Notepad++ window titled 'C:\Python39\ex3\_5.py - Notepad++'. The code in the editor is:

```
1 name = input('Please enter your name: ')
2 print(f'Hi, {name}.')
```

The console window below shows the execution output:

```
C:\Python39\python.exe "C:\Python39\ex3_5.py"
Process started (PID=10860) >>>
Please enter your name: Chris
Hi, Chris.
<<< Process finished (PID=10860). (Exit code 0)
===== READY =====
```

The status bar at the bottom indicates: Python file, length: 64, lines: 2, Ln: 2, Col: 22, Sel: 0 | 0, Windows (CR LF), UTF-8, INS.

**Figure 3-2.** User input, saying "hi"

### Explanation



Reiterate `ex3_4.py` and write some simplified code in `ex3_5.py`.

- ① The number of code lines has shrunk. In this exercise, you are learning how to reduce two code lines into a single line of code and also use an abbreviated formatting method.

### Exercise 3-6: Get a Username: Version 2

On Notepad++, enter the following code, save the file as `ex3_6.py` and run the code. (see Figure 3-3).

- ```
#Ex3_6.py

import re

name = input("Enter your name: ")

while True:
    ① while not re.match("^[a-zA-Z]+$", name):
        name = input("Enter your name: ")
    else:
        print(name)
        exit()
```

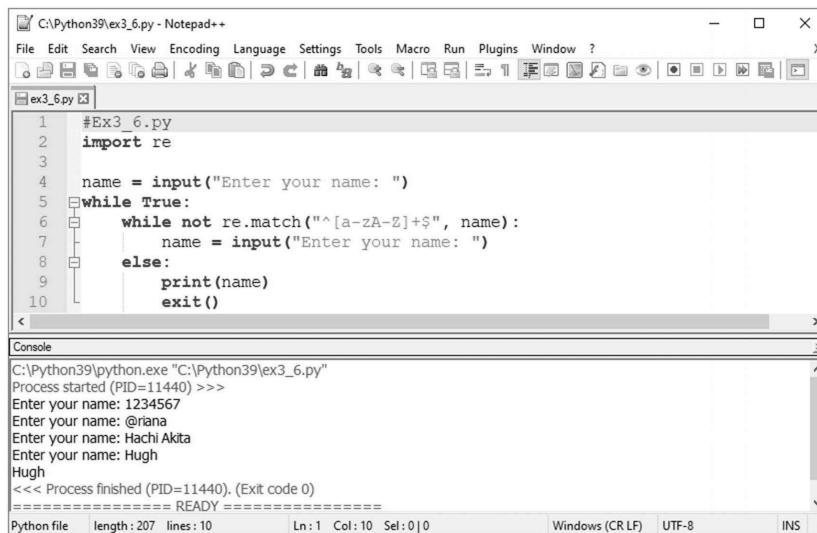


Figure 3-3. User input, getting correctly formatted input

Explanation

- ① Using `ex3_5.py` as the basis, you have just created a simple Python application asking for someone's name with some requirements. You have imported the `re` (regular expression) built-in Python module to force the user to provide a name starting with an alphabetical letter that accepts letters only. `^[a-zA-Z]+$` means any character starting with a letter with one or more characters and ending with a letter. So, you cannot enter a number or a special character as a name. If the user

cannot adhere to this rule, the script asks until the user provides an expected response. Once the correct response is received, the name is printed and exits the application .

You will learn more about regular expressions in Chapter 9. An entire chapter will be dedicated to learning regular expressions and how to use the `re` module. Scripting is all about data handling and how we process them. We must master regular expressions and conquer our fear of regular expressions to become confident programming engineers.

Exercise 3-7: Get a Username: Version 3

Enter the code and save the file as `ex3_7.py` on Notepad++. Run the code as shown in Figure 3-4.

```
# ex3_7.py
import re

def get_name():
    1name = input("Enter your name: ")
    1while True:
    ① 2while not re.match("[a-zA-Z]+$", name):
    3name = input("Enter your name: ")
    2else:
    3print(name)
    3exit()
    get_name()
```



Figure 3-4. User input, getting the username script into a function

Explanation

Create the `ex3_7.py` script and run it to learn how you can create a function based on `ex3_6.py` and convert it into a function.

- ① The last statement, `get_name()`, will trigger the script to run. This is the same script as `ex3_6.py`, but we have converted it into a function so you can understand the reiteration process.

Exercise 3-8: Add a Temporary Filepath, Import `ex3_7.py` as a Module, and Run the Script

- ① `C:\Users\brendan>python`
 Python 3.9.1 (tags/v3.9.1:1e5d33e, Dec 7 2020,
 17:08:21) [MSC v.1927 64 bit (AMD64)] on win32

Type "help", "copyright", "credits" or "license" for more information.

```
>>> import ex3_7.py
```

Enter your name: **Brendan**

Brendan

C:\Users\brendan>

```
# If your script is under a different folder, for example, under 'C:\Users\brendan\Documents\'. You have to manually add the Filepath using append.
```

```
>>> import sys
```

```
>>> sys.path.append('C:\\Users\\brendan\\Documents\\')
```

```
# Add temporary filepath
```

```
>>> print(sys.path)
```

```
② [' ', 'C:\\Python39\\python39.zip',
   'C:\\Python39\\DLLs', 'C:\\Python39\\lib',
   'C:\\Python39', 'C:\\Python39\\lib\\site-packages',
   'C:\\Users\\brendan\\Documents\\']
```

```
>>> import ex3_7.py
```

9

Enter your name: Ryan

Ryan

Explanation

If you selected Add Python 3.9 to PATH during the Python installation, you should be able to call the Python script as a module

① and run it from anywhere on your Windows operating system.

If you forget, you can choose to add a temporary filepath, as shown in example ②.

Open the Python command prompt and add a temporary filepath so Python can read the Python module created in

② Exercise 3-7. Here you are adding a new (temporary) filepath and importing a custom module to run from your interpreter session to learn how importing and customizing modules work.

Exercise 3-9: Use Commas to Add Spaces Between Strings

```
① >> print('Around' + 'the' + 'World' + 'in' + '100' +
        'days.')
```

AoundtheWorldin100days. # by default, no spaces are added when '+' is used

```
>>> print('Around ' + 'the ' + 'World ' + 'in ' + '100
        ' + 'days.') # manually enter spaces between words
```

```
Around the World in 100 days .
>>> print('Around', 'the', 'World', 'in', '100',
'days.')
Around the World in 100 days. # adds spaces
automatically
```

Explanation

- Practice the previous exercises and compare the output. Notice
- ① that commas are used in the previous exercise, and the output adds spaces between the strings.

Exercise 3-10: Practice if; if and else; and if, elif, and else

```
① >>> y = 5
>>> if x > y:
...     print('x is greater than y.')
...
>>> # No output as if the statement was False
>>> x = 3
>>> y = 5
>>> if x < y:
...     print('x is less than y.')
...
x is less than y. # prints output as if the statement
was True
>>> y = 5
>>> if x >= y:
...     print('x is greater or equal to y.')
... else:
...     print('x is smaller than y.')
...
x is smaller than y. # prints output of else
statement
>>> x = 3
>>> y = 5
>>> z = 3
>>> if x < y > z:
...     print('y is greater than x and greater than z.')
...
y is greater than x and greater than z. # all condi-
tions met
>>> x = 5
>>> y = 10
```

```
>>> if x < y:
...     print('x is smaller than y.')
... elif x > y:
...     print('x is greater than y.')
... else:
...     print('x is equal to y.')
...
x is smaller than y. # if the statement was satisfied
```

Explanation

You have just practiced `if`; `if` and `else`; and `if`, `elif`, and `else`.

- ① These exercises are self-explanatory; this is one beauty of Python.

Exercise 3-11: Practice for `~` in `range` with `end=''`

```
① >>> for n in range (1, 11):
...     print(n)
...
1
2
3
[omitted for brevity]
9
10

>>> for n in range (1, 11):
...     print(n, end='')
...
12345678910

>>>
>>> for n in range (1, 11):
...     print(n, end=' ') # adds spaces and print
horizontally
...
1 2 3 4 5 6 7 8 9 10

>>>
>>> for n in range (1, 11):
...     print(n, end=',') # adds commas between output
...
1,2,3,4,5,6,7,8,9,10,

>>>
>>> for n in range (1, 11):
...     print(n, end='/') # adds forward slashes
```

```
...
1/2/3/4/5/6/7/8/9/10/
>>>
>>> for n in range (1, 11):
...     print(n, end='|') # adds pipes (vertical bars)
...
1|2|3|4|5|6|7|8|9|10|
>>>
```

Explanation

- ① Practice each `for ~ in range` statement with `end=` and compare the variations in the outcome.

Exercise 3-12: Practice for ~ in range

```
>>> for n in range (2, 11):
...     print("Creating VLAN" + str(n))
...
① Creating VLAN2 Creating VLAN3 Creating VLAN4 Creating
VLAN5 Creating VLAN6 Creating VLAN7 Creating VLAN8
Creating VLAN9 Creating VLAN10
```

Explanation

- ① `for ~ in range()` becomes handy when you have to loop through several lines to add configuration to networking devices. We will apply the previous code in the later Python network automation labs . Try to get familiar with this example.

Exercise 3-13: Practice for line in ~

```
① >>> txt1 = "Seeya later Alligator"
>>> for line in txt1:
...     print(line)
...
S
e
e
[omitted for brevity]
o
r
>>> txt1 = "Seeya later Alligator"
```

```
>>> for line in txt1:
...     print(line, end='')
...
Seeya later Alligator
```

Explanation

For strings, you can use the `for line in ~ method`, and you

- ① will get the same effect as the `for n in range()` method for numbers.

Exercise 3-14: Use the `split()` Method

```
>>> txt1 = "Seeya later Alligator"
>>> print(txt1.split()) # splits at every white spaces
['Seeya', 'later', 'Alligator']
>>> print(txt1.split( )) # same as above
['Seeya', 'later', 'Alligator']
>>> print(txt1.split('e', 1))
['S', 'eya later Alligator'] #splits at first instance
of 'e'
>>> txt2 = (txt1.split('e', 1))
① >>> print(txt2[0]) # you only want to extract the
letter 'S'
S
>>> txt3 = (txt1.split('a', 3))
>>> print(txt3) # splits at every 'a', three times
['Seey', ' l', 'ter Allig', 'tor']
>>> print(txt3[1]) # you only want to extract the
first letter 'l'
L
```

Explanation

Getting enough practice with string indexing and splicing becomes a powerful tool when used with regular expressions.

- ① Unfortunately, to become a good programmer, we have to get familiar with general data handling: data gathering, data processing, and data flow control.

Exercise 3-15: Practice with `lstrip()`, `rstrip()`, `strip()`, `upper()`, `lower()`, `title()`, and `capitalize()`

```

>>> australia = ' terra australis incognita '
>> print(australia.title()) # title() method capital-
izes first letters
Terra Australis Incognita
>>> print(australia.rstrip().upper()) # upper() for
capitalization
TERRA AUSTRALIS INCOGNITA
① >>> x = (australia.lstrip().lower()) # lower() for
lower casing
>>> print(x.capitalize()) # capitalize() for capital-
ization of first letter only
Terra australis incognita
>>> print(australia.strip().upper()) # strip() remove
both left and right white spaces
TERRA AUSTRALIS INCOGNITA

```

Explanation

- ① You have just completed the uppercasing and lowercasing and strip method practice. Can you think of some interesting and relatable words for you to practice? Add some fun into your exercise by changing the words to have more meaning to you.

Exercise 3-16: Create a file and read it four different ways

```

>>> with open('test123.txt', 'w') as f:
...     f.write('This is line 1.\nThis is line 2.\nThis
① is line 3.\n')
...
48

```

```

>>> with open('test123.txt', 'r') as f:
...     lines = f.readlines()
...     print(lines)
② ...
['This is line 1.\n', 'This is line 2.\n', 'This is
line 3.\n']

```

```

③ >>> with open('test123.txt', 'r') as f:
...     lines = list(f)
...     print(lines)
...

```



```
['This is line 1.\n', 'This is line 2.\n', 'This is
line 3.\n']
```

```
>>> f = open('test123.txt', 'r')
>>> lines = f.readlines()
>>> print(lines)
④ ['This is line 1.\n', 'This is line 2.\n', 'This is
line 3.\n']
>> f.close()
```

```
>>> f = open('test123.txt', 'r')
>>> lines = list(f)
>>> print(lines)
⑤ ['This is line 1.\n', 'This is line 2.\n', 'This is
line 3.\n']
f.close()
```

Explanation

① Create `test123.txt` in `w` mode and write something.

② This is with the `open` method with a `readlines()` example.

③ This is with the `open` method with a `list()` example.

④ This is a simple `open()` method with a `readlines()` example.
Always close the file after use.

⑤ This is a simple `open()` method with a `list()` example. Make
sure `f.close()` is issued to close the file.

Exercise 3-17: Read and Output Files for a More Detailed Understanding

```
>>> with open('test123.txt', 'w') as f:
...     f.write(' this is a lower casing line.\n')
...
33
① >>> with open('test123.txt', 'r') as f:
...     print(f.read())
...
this is a lower casing line.
```

```
② >>> with open('test123.txt', 'a') as f:
```

```
...     f.write('THIS IS AN UPPER CASING LINE.
\nThisIsACamelCasingLine.\n')
...
58
```

```
>>> with open('test123.txt', 'r') as f:
...     print(f.read())
...
this is a lower casing line. THIS IS AN UPPER CASING
LINE.
ThisIsACamelCasingLine .
>>> with open('test123.txt', 'r') as f:
③ ...     print(f.readline())
...
this is a lower casing line.
>>> with open('test123.txt', 'r') as f:
...     print(f.readlines())
...
[' this is a lower casing line. \n', 'THIS IS AN
UPPER CASING LINE .\n', 'ThisIsACamelCasingLine.\n']
```

```
>> >>> with open('test123.txt', 'r') as f:
...     print(f.read().lower().strip())
④ ...
this is a lower casing line. this is an upper casing
line. thisisacamelcasingline.
```

```
>>> with open('test123.txt', 'r') as f:
...     x = (f.read().lower().strip())
...     y = set(x.split())
...     print(y)
⑤ ...     print(len(y))
...
{'lower', 'casing', 'a', 'this', 'is', 'upper', 'thi-
sisacamelcasingline.', 'an', 'line.'} 9
```

Explanation

Create a new `test123.txt` file. Since the file opens in `w` mode, it

- ① will overwrite the file created in Exercise 3-17. Write to the file in the same way as shown in this book.

- ② You can write more content to your file in a mode .

- ③ Try to read and print in three different modes: `read()`, `readline()`, and `readlines()`. Do you see the difference?
- ④ Practice using other methods to manipulate your strings and whitespaces.
- ⑤ You are using the `set()` and `split()` methods together to count the total number of words and characters in your file.

Exercise 3-18: Use the `getpass()` Module and User Input

For this exercise, open Python IDLE (interpreter) to write the code.

```
>>> import getpass
>>> def get_pwd():
...     username = input('Enter username : ')
...     password = getpass.getpass()
...     print(username, password)
...
>>> get_pwd()
Enter username : admin
Password: *****
admin mypassword
```

Explanation

- ① Use `import` to load the `getpass` library. Use `def` to change your password into a function. Then use the `input()` function to ask for the user's name; then use the `getpass.getpass()` module to ask for the user's password. While the password is entered, the `getpass()` module will hide the entered password. Just for testing, we have printed out the username and password. The `getpass()` module comes in handy when creating a login request for SSH and Telnet to networking devices.

Exercise 3-19: Understand the Difference Between Encoding and Decoding

```
>>> text_1 = 'Network Automation'
>>> print(text_1)
Network Automation
① >>> byte_1 = text_1.encode()
>>> print(byte_1)
b'Network Automation '
```

```
>>> byte_2 = b'Mission completed.'
>>> print(byte_2)
b'Mission completed. '
② >>> text_2 = byte_2.decode()
>>> print(text_2)
Mission completed.
```

Explanation

Computers communicate in bits (0s and 1s), and the computer handles files in bytes. But we humans want to communicate in plaintext to the machines, so encoding and decoding are required. This is a simple exercise to convert a string into bytes. You will see this in action during the Telnet Python lab exercises in Chapter 13. Make a special note of this exercise.

② In Python 3, all strings are recognized as Unicode, and the `decode()` method must be used to convert bytes, which are binary stream data that computers understand. When executing commands on machines such as routers and switches, be wary of bytes to string conversion and string to bytes conversion to avoid code execution errors.

Exercise 3-20: Handle CSV Files in Python with the csv Module

```
① # Use the following information to create .py called ex3_20.py and save it to your Desktop folder.
# ex3_20.py
import csv
with open ('C://Users//brendan//Documents//2020_router_purchase.csv', 'w', newline='') as csvfile:
    filewriter = csv.writer (csvfile, delimiter = ',', quotechar = '|', quoting = csv.QUOTE_MINIMAL)
    filewriter.writerow (['Site', 'Router_Type', 'IOS_Image', 'No_of_routers', 'Unit_price', 'Purchase_Date'])
    filewriter.writerow(['NYNY', 'ISR4351/K9', 'isr4300-universalk9.16.09.05.SPA.bin', 4, 9100.00, '1-Mar-20'])
    filewriter.writerow(['LACA', 'ISR4331/K9', 'isr4300-
```

```

universalk9.16.09.05.SPA.bin', 2, '$ 5162.00', '1-Mar- 20'])

filewriter.writerow(['LNUK', 'ISR4321/K9', 'isr4300-universalk9.16.09.05.SPA.bin', 1
2370.00', '3-Apr- 20'])

filewriter.writerow(['HKN', 'ISR4331/K9', 'isr4300-universalk9.16.09.05.SPA.bin', 2
5162.00', '17-Apr-20'])

filewriter.writerow(['TKJP', 'ISR4351/K9', 'isr4300-universalk9.16.09.05.SPA.bin', 1
9100.00', '15-May-20'])

filewriter.writerow(['MHGM', 'ISR4331/K9', 'isr4300-universalk9.16.09.05.SPA.bin', 2
5162.00', '30-Jun-20'])

```

Open the Windows command prompt or Windows PowerShell, and run the Python code to create your file (see Figure 3-5).

```

PS C:\Users\brendan> cd Documents # change directory to Documents
② PS C:\Users\brendan\Documents> dir ex3_20.py # check ex3_20.py exists under Documents
PS C:\Users\brendan\Documents> python ex3_20.py # run ex3_20.py to create a csv file
PS C:\Users\brendan\Documents> dir 2020_router_* # check 2020_router_purchase.csv file
created

```

③ Use Microsoft Excel to open the file and check the newly created file. It should look similar to Figure 3-6.

```

# For Windows users
>>> f = open('C://Users//brendan//Documents//2020_router_purchase.csv', 'r')
>>> f
<_io.TextIOWrapper name='C://Users//brendan//Documents//2020_router_purchase.csv' mode='r' encoding='cp1252'>
④
# For Linux users (optional)
>>> f = open ('2020_router_purchase.csv', 'r')
>>> f
<_io.TextIOWrapper name = '2020_router_purchase.csv' mode = 'r' encoding = 'UTF-8'>

```

```

⑤ >>> f = open('C://Users//brendan//Documents//2020_router_purchase.csv', 'r')
>>> routers = f.read()
>>> routers
'Site,Router_Type,IOS_Image,No_of_routers,Unit_price($),Purchase_Date\nNYNY,ISR4351/I
universalk9.16.09.05.SPA.bin,4,$ 9100.00,1-Mar-20\nLACA,ISR4331/K9,isr4300-
universalk9.16.09.05.SPA.bin,2,$ 5162.00,1-Mar-20\nLNUK,ISR4321/K9,isr4300-
universalk9.16.09.05.SPA.bin,1,$ 2370.00,3-Apr-20\nHKN,ISR4331/K9,isr4300-
universalk9.16.09.05.SPA.bin,2,$ 5162.00,17-Apr-20\nTKJP,ISR4351/K9,isr4300-
universalk9.16.09.05.SPA.bin,1,$ 9100.00,15-May-20\nMHGM,ISR4331/K9,isr4300-
universalk9.16.09.05.SPA.bin,2,$ 5162.00,30-Jun-20\n'
>>> print(routers)
Site,Router_Type,IOS_Image,No_of_routers,Unit_price($),Purchase_Date
NYNY,ISR4351/K9,isr4300-universalk9.16.09.05.SPA.bin,4,$ 9100.00,1-Mar-20
LACA,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$ 5162.00,1-Mar- 20
LNUK,ISR4321/K9,isr4300-universalk9.16.09.05.SPA.bin,1,$ 2370.00,3-Apr- 20

```

```

HKCN,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$ 5162.00,17-Apr-20
TKJP,ISR4351/K9,isr4300-universalk9.16.09.05.SPA.bin,1,$ 9100.00,15-May-20
MHGM,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$ 5162.00,30-Jun-20

>>> f.close ()

```

```

>>> with open('C://Users//brendan//Documents//2020_router_purchase.csv', 'r') as f:
...     routers = f.readlines()
...     for router in routers:
...         print(router, end=" ")
...

```

⑥

```

Site,Router_Type,IOS_Image,No_of_routers,Unit_price($),Purchase_Date
NYNY,ISR4351/K9,isr4300-universalk9.16.09.05.SPA.bin,4,$ 9100.00,1-Mar-20
LACA,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$ 5162.00,1-Mar- 20
LDUK,ISR4321/K9,isr4300-universalk9.16.09.05.SPA.bin,1,$ 2370.00,3-Apr- 20
HKCN,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$ 5162.00,17-Apr-20
TKJP,ISR4351/K9,isr4300-universalk9.16.09.05.SPA.bin,1,$ 9100.00,15-May-20
MHGM,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$ 5162.00,30-Jun-20

```

The screenshot shows a Windows PowerShell window with the following commands and output:

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\brendan> cd Documents
PS C:\Users\brendan\Documents> dir ex3_20.py

    Directory: C:\Users\brendan\Documents

Mode                LastWriteTime         Length Name
----                -
-a----           29/12/2020   4:33 PM           1062 ex3_20.py

PS C:\Users\brendan\Documents> python ex3_20.py
PS C:\Users\brendan\Documents> dir 2020_router_*

    Directory: C:\Users\brendan\Documents

Mode                LastWriteTime         Length Name
----                -
-a----           29/12/2020   4:34 PM           528 2020_router_purchase.csv

PS C:\Users\brendan\Documents>

```

Figure 3-5. Running the CSV script to create an Excel file on Windows PowerShell

	A	B	C	D	E	F	G
	Site	Router_Type	IOS_Image	No_of_routers	Unit_price(\$)	Purchase_Date	
1	NYNY	ISR4351/K9	isr4300-universalk9.16.09.05.SPA.bin	4	\$9,100.00	1-Mar-20	
2	LACA	ISR4331/K9	isr4300-universalk9.16.09.05.SPA.bin	2	\$5,162.00	1-Mar-20	
3	LDUK	ISR4321/K9	isr4300-universalk9.16.09.05.SPA.bin	1	\$2,370.00	3-Apr-20	
4	HKCN	ISR4331/K9	isr4300-universalk9.16.09.05.SPA.bin	2	\$5,162.00	17-Apr-20	
5	TKJP	ISR4351/K9	isr4300-universalk9.16.09.05.SPA.bin	1	\$9,100.00	15-May-20	
6	MHGM	ISR4331/K9	isr4300-universalk9.16.09.05.SPA.bin	2	\$5,162.00	30-Jun-20	

Figure 3-6. Checking 2020_router_purchase.csv in Excel

Explanation

- ① Create a new code file called `ex3_20.py` using the information .
The file needs to be saved in your `Documents` folder.

- ② Then open a command prompt of your choice and run the Python command to create a CSV file using the `ex3_20.py` script.

- ③ Open the newly created `2020_router_purchase.csv` file in Microsoft Excel to check it.

- ④ If you open and read the file, you can see that the Windows-created file will use the encoding of `cp1252`, whereas on Linux, UTF-8 encoding will be used as the default decoding method.

- ⑤ Use `read()` to read the file . The content is a long string of data.
If you use `print()`, then the new lines will be displayed as individual lines, and it becomes easier to read.

- ⑥ If you do not want to be bothered with the ending `f.close()` statement , you can also use the `with open` command to open the file and read it. In this example, we are using the `f.readlines()` method to read each line separately and then using the `for` loop to call out each line in the CSV file. Take your time carefully studying the difference between the `read()`, `readlines()`, and `readline()` methods.

Exercise 3-21: Output a CSV File

- ①

```
>>> f =
open('C://Users//brendan//Documents//2020_router_purchase.csv', 'r')
>>> for line in f:
...     print(line.strip())
```

```
...
Site,Router_Type,IOS_Image,No_of_routers,Unit_price($),Purchase_Date
NYNY,ISR4351/K9,isr4300-universalk9.16.09.05.SPA.bin,4,$ 9100.00,1-
Mar-20 LACA,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$
5162.00,1-Mar-20 LDUK,ISR4321/K9,isr4300-
universalk9.16.09.05.SPA.bin,1,$ 2370.00,3-Apr-20
HKCN,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$ 5162.00,17
-Apr-20
TKJP,ISR4351/K9,isr4300-universalk9.16.09.05.SPA.bin,1,$ 9100.00,15
-May-20
MHGM,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$ 5162.00,30
-Jun-20
>>> f.close()
```

Explanation

- As in a normal text file, you can use the `for` loop method to read each line and print one line at a time. Make sure you use the `strip()` method to remove any leading or trailing whitespaces, newline, or tabs.

Exercise 3-22: Find the Price of a Cisco ISR 4331 Router from a CSV File

```
① >>> f = open('C://Users//brendan//Documents//2020_router_purchase.csv',
#1 'r')
#2 >>> x = f.read().split('\n')
#3 >>> x
#4 ['Site,Router_Type,IOS_Image,No_of_routers,Unit_price($),Purchase_Date',
#5 'NYNY,ISR4351/K9,isr4300- universalk9.16.09.05.SPA.bin,4,$ 9100.00,1-
Mar-20', 'LACA,ISR4331/K9,isr4300- universalk9.16.09.05.SPA.bin,2,$
5162.00,1-Mar-20', 'LDUK,ISR4321/K9,isr4300-
universalk9.16.09.05.SPA.bin,1,$ 2370.00,3-Apr-20',
'HKCN,ISR4331/K9,isr4300- universalk9.16.09.05.SPA.bin,2,$ 5162.00,17 -
Apr-20', 'TKJP,ISR4351/K9,isr4300- universalk9.16.09.05.SPA.bin,1,$
9100.00,15 -May-20', 'MHGM,ISR4331/K9,isr4300-
universalk9.16.09.05.SPA.bin,2,$ 5162.00,30 -Jun-20', '']
>>> y = x[2]
>>> y
'LACA,ISR4331/K9,isr4300-universalk9.16.09.05.SPA.bin,2,$ 5162.00,1-Mar-
20'
>>> z = y.split(',')
>>> z
```



```
['LACA', 'ISR4331/K9', 'isr4300-universalk9.16.09.05.SPA.bin', '2', '$
5162.00', '1-Mar-20']
>>> price_4331 = z[4]
>>> print(price_4331)
$ 5162.00
```

Explanation

In this example, you have just practiced reading data from a CSV file and extracting specific information.

- #1. The `read().split()` method reads the file, splits the items, and saves the items in a list for each line.
- #2. `x`'s index 2 was assigned to variable `y`.
- ① #3. Once again, `split()` breaks the string into a list. The list was assigned to a variable `z`.
- #4. `z`'s fifth or index 4 elements were assigned to a variable called `price_4331`.
- #5. The price of a Cisco 4331 was printed on the screen.

Exercise 3-23: Calculate the Total Cost of the Router Purchases: No Module

```
# ex3_23.py
total = 0.0

with open
('C://Users//brendan//Documents//2020_router_purchase.csv',
'r') as f:
    headers = next (f)
    for line in f:
        line = line.strip ()
        devices = line.split(',')
        devices[4] = devices[4].strip('$')
        devices[4] = float (devices[4])
        devices[3] = int (devices[3])
        total += devices[3] * devices[4]
    print('Total cost: $', total)
```

```
② C:\Users\brendan\Documents> python ex3_23.py
Total cost: $ 78842.0
```

Explanation

- ① Using the method learned in Exercise 3-23, create a program that calculates the total cost of routers called `ex3_23.py`. We set

the initial value of the total at 0.0 dollars. Use indexes 3 and 4 in the list to find the sum spent on the router purchase. To avoid errors that can occur during calculations, we use `header=next(f)` to skip the first line's header information. Split, strip, and convert between various data types. Save the script in the `Document` folder, and make sure you change the file path to suit your computer settings.

- Run the Python application to find the total cost of the router purchases. According to our script, the total cost was \$78,842.
- ② These exercises seem very anal and meaningless but will become useful while processing meaningful data and developing real work applications.

Exercise 3-24: Calculate the Total Cost of the Router Purchases: Using the csv Module

```
# ex3_24.py
import csv
total = 0.0
with open
('C://Users//brendan//Documents//2020_router_purchase.csv')
as f:
    rows = csv.reader (f)
    headers = next (rows)
    for row in rows:
        row [4] = row [4].strip ('$')
        row [4] = float(row [4])
        row [3] = int(row [3])
        total += row[3] * row[4]
print('Total cost: $', total)
```

② C:\Users\brendan\Documents> **python ex3_24.py**
Total cost: \$ 78842.0

Explanation

- We re-created the same application using Python's built-in `csv` module. First, import the `csv` module and follow the same
- ① process of striping and converting between data types. There is a famous saying in programming, "There's more than one way to skin a cat."

- ② Run the new code, but the total cost will be the same.

As we discussed earlier, this chapter only covers basic and specific Python topics. I recommend you pick up other basic Python books and go deeper into the topics discussed in this book. Well done! You have completed the basic Python exercises to get you ready for some Python network automation, but more advanced topics will follow in the later chapters.

Exercise 3-25: Convert dd-mmm-yy and Calculate the Difference in Days and Then in Years

While working in networking, you'll notice that different vendors have different date and time formats in their software. One good example is the date format of Cisco routers and switches. While I was manipulating the data collected from actual Cisco IOS software, I had to spend an hour converting and manipulating the release date of Cisco IOS in a particular format and work out how old the IOS image was since its release date. This exercise is from a real production example script. Here's the format of the IOS release date given: 12-Jul-17.

The objective is to manipulate and convert the date to the correct format so Python can work out how old the IOS is since its release date. Note, your date will differ from the result given in this example as it uses `today()` as the calculation date. Follow along with example ① first and then follow through with the iteration example ②.

```
# Follow this exercise in Python IDLE

>>> from datetime import datetime
>>> IOS_rel_date = '12-Jul-17'
① >>> x = IOS_rel_date.replace('-', ' ')
#1 >>> y = datetime.strptime(x, '%d %b %y')
#2 >>> y
#3 datetime.datetime(2017, 7, 12, 0, 0)
#4 >>> y.date() datetime.date(2017, 7, 12)
#5 >>> y_day = y.date()
#6 >>> datetime.today()
#7 datetime.datetime(2020, 4, 15, 11, 56, 3, 370515)
#8 >>> datetime.today().date()
#9 datetime.date(2020, 4, 15)
#10 >>> t_day = datetime.today().date()
#11 >>> delta = t_day - y_day
#12 >>> print(delta.days)
#13 1008

>>> years = round(((delta.days)/365), 2)
2.76
```

```
② # Reiterated and reduced version of example 1:
>>> from datetime import datetime
```

```
>>> x = IOS_rel_date.replace('-', ' ')
>>> y_day = (datetime.strptime(x, '%d %b %y')).date()
>>> t_day = datetime.today().date()
>>> delta = t_day - y_day
>>> years = round((delta.days)/365, 2)
>>> print(years)
2.76
```

Explanation

#1. Import the `datetime` module from the `datetime` library. We need this library to complete our challenge.

#2. Enter the IOS release date as a string to begin this exercise.

#3. Replace `-` with blank spaces .

#4. Use the `datetime` module's `strptime` method to convert `x` to the correct date format. `%d` represents days, `%b` represents abbreviated months such as JAN, Jun, and Jul, and `%y` represents the abbreviated year without the first two leading digits.

Python's `datetime` module expects the date to be in a specific format, and we are trying to normalize the format to achieve our goal.

#5. Now type `y` to check the current format. It has two 0s separated by a comma, which is the hour and minute. We have to lose these two zeros.

#6. Now use the `date()` method on `y` to drop the hour and minute information. Check first before applying this. This is the date format required to solve half of our challenge. We are at 50 percent now.

①

#7. Now we are happy with the date format, so assign it to the variable `y_day`.

#8. It is time to work with today's date. To find the number of days since the IOS release, we need to work out what today is and then minus the day of the release date. Use the `datetime()` module again, this time on `today()`.

#9. If the result is acceptable, then append `.date()` at the back to drop the time elements. This will return the date in the correct format.

#10. Now assign the object from #9 to a variable, `t_day`.

#11. The delta or the difference between the two dates is `t_day - y_day`.

#12. Print the result, and it will return the delta in the number of days.

#13. Now divide the delta by 365 days to convert the days to years. Use the `round()` function to give the last two decimals.

This single exercise will give you some insight into how to massage specific data, in this case, dates.

②

This example is a reiterated version of example 1. I have crunched some functions into one line to shorten the conversion processes. This is just one example of data manipulation, and there are many other ways to do the same conversion. Still, the important thing is to understand the thought process so you can adapt it to your situation. Finally, if you have 10 minutes, please visit the following site to review what [strftime](https://strftime.org/) code formats are available: <https://strftime.org/>.

Exploring the Python IDE Environment

After completing this chapter, you need to start thinking about an integrated development environment (IDE) for Python code development. Working in a text editor is straightforward, but it has its limitations. Working within an IDE is not compulsory, but as you write more complex and longer Python code, your coding efficiency drops when using text editors such as Notepad, EditPlus, or Notepad++ on Windows or vi, Emacs, Gedit, or nano on Linux. IDEs provide many features not available from standard text editors. When you work in collaboration with team members, suddenly the version control of your documents becomes essential. Many organizations use GitHub with cloud storage as their code version control solution. Still, you do not have to worry about GitHub or version control at this stage. You have to learn to crawl before you can walk. Hence, this topic is not discussed beyond this section in this book. You do not have to start using an IDE application in this book, but I encourage you to explore them one at a time and find the perfect match for your coding style. On enterprise Linux systems, you rarely see graphical user interfaces (GUIs) , so only Windows-compatible IDEs are discussed here. To help you explore various Python IDEs available for Windows, see Table 3-1. Although Jupyter Notebook (Anaconda) is not an IDE, it is included in Table 3-1 to give you another path to studying Python.

Table 3-1. Python IDEs and Anaconda (Jupyter Notebook)

Recommended IDE	Pros and Cons, Download URL
PyCharm*	Pros: Feature-rich, intuitive user interface, supports cross-platform, made for Python, free Community version, friendly for both beginners and experts Cons: Paid Professional version URL: https://www.jetbrains.com/pycharm/download/
Microsoft Visual Studio	Pros: Windows-friendly , easy to use (especially if you are a Windows user), feature-rich, Community version free Cons: The interface can be slow to start up. URL: https://visualstudio.microsoft.com/vs/features/python/
Eclipse	Pros: Multiple programming language support, familiar user interface, free

Recommended IDE	Pros and Cons, Download URL
	Cons: JRE (JDK) dependency, tricky to set up for beginners, older IDE URL: https://www.eclipse.org/downloads/packages/installer
Atom	Pros: Developed by GitHub's Electron text editor, supports cross-platform and multiple programming languages, feature-rich via extensions and plug-ins, good documentation on application programming interface (API), free, easy to use, light yet still powerful Cons: Not as intuitive as PyCharm, closer to a text editor than PyCharm URL: https://atom.io/
Sublime Text 3	Pros: Popular code editor with fast, stable, and mature user interface; many features with modular approach and extendability Cons: Not as versatile as other IDEs, still feels more like a text editor than an IDE, have to purchase a license for continued use, no debugging option URL: https://www.sublimetext.com/3
Anaconda – Jupyter Notebook	Pros: Beginner friendly, excellent Python learning tool, web-based, used by scientific communities for data analytics Cons: Not an IDE, web-based, system resource-heavy URL: https://jupyter.org/install

Summary

This chapter was brief but contained a lot more substance than Python's `print()` function and syntax. This chapter served as a test-drive chapter that gives you the feel for what is ahead of us in the next chapter; you learned more Python basics by looking at some new Python modules, handling files, and processing data using simple Python code. Chapters [2](#) and [3](#) should serve as a good foundation for the Python network automation labs to be presented later in this book. If you still do not fully comprehend some parts of the exercise, it is OK to go back and repeat the exercises two to three times. If you have the time, you should repeat the exercises in both chapters three times by the time you get to Chapter [11](#). Chapters [4](#) to [6](#) will focus on building the virtually integrated Python network automation labs, starting with VMware Workstation Pro installation. It's time to move on to Chapter [4](#) to learn VMware Workstation basics and general virtualization technology basics. VMware Workstation

and its virtualization technologies will help you to build your complete virtual networking lab and serve as the glue holding together virtual machines in your lab, so I encourage you to read the next chapter in its entirety.

Storytime 2: Machine vs. Human, The First Confrontation



According to Wikipedia, the first machine versus human confrontation took place in the United States of America around the 1870s, when a miner named John Henry challenged an automatic steam drilling machine to make a railway tunnel. The story of John Henry has been passed down to younger generations in American nursery rhymes. In the olden days, to drill a railway tunnel, the railway miners had to use hammers, chisels, and drills to make holes and use dynamite to blow up the land-mass to make a tunnel. This human to machine competition became so famous in America that the story of John Henry versus the steam engine became an American folk legend. According to the story, the competition for tunnel boring went on for a day and a half to bore dynamite holes and lay the tracks.

According to the folk story, John Henry beat the steam drill and was victorious while defending his job. Sadly, according to the story, John Henry died of a heart attack due to fatigue and the severe stress after the competition. Of course, the steam drill continued to drill the holes the next day and the following days. What is clear here is that John Henry and his colleagues' jobs were getting automated out, and the U.S. Railroad Corporation did not hesitate to replace the miners with machines. The

same thing happened in farming and manufacturing during the first Industrial Revolution, and automation of human tasks continues today and will continue in the future.

According to Wikipedia, “Automation is the technology by which a process or procedure is performed with minimal human assistance.” We all agree that automation generally improves human life’s quality, but there are always two sides to the automation coin. John Henry was a simple miner, digging holes for a living with little education in machinery or machine operations. It would have been challenging for him to learn how to operate the heavy machinery, the steam drill, which was the latest and greatest technology in the 1870s. So, transitioning from an old job to a new job would have been almost impossible for the poor fellow. Looking at today’s automation and IT automation, in particular, the existing engineers are already finding themselves in the same situation as John Henry, and more will face the music in the future as IT technologies are evolving rapidly. The advancement in IT technologies will replace many existing IT jobs. Still, the good news is that new technologies will also create new jobs and opportunities for those who keep learning to survive our cutting-edge IT industry. If your work gets automated and you find yourself out of your job, you will feel miserable for losing your job, but you may also enjoy it if you have been preparing to transition to IT automation.

Source: https://en.wikipedia.org/wiki/John_Henry__ (folklore)

Table of contents collapsed